

1. Introduction to Node – presented on 23 Januari 2024

1. Buka dokumentasi Node <https://nodejs.org/en/docs/>
2. Download dan install Node
3. Buka command prompt dan ketik `node -v`

2. Memahami npm: Node Package Manager

Module

1. Buatlah folder baru **nodejs** di **C:\Users\Kurnia\Desktop** (disesuaikan).
2. Buat file menggunakan notepad++ dan namakan dengan **my-module.js**.
3. Lalu buat satu lagi file dengan nama **module-demo.js**.
4. Simpan kedua file tersebut di folder **C:\Users\Kurnia\Desktop\nodejs**
5. Disini kita akan memahami konsep *code reuse*. Code bisa diakses dari satu file ke file lainnya.
6. Ketik code berikut pada file **my-module.js**:
`exports.myText = "Pesan dari file my-module"`
7. Ketik code berikut pada file **module-demo.js**:
`var myModule = require('./my-module.js')
console.log(myModule.myText)`
8. Ketik **node module-demo.js** pada command prompt.
9. Tambahkan code berikut pada file module-demo.js:
`var h = "selamat pagi";
var slc = h.slice(4);
console.log(`ucapan ${slc}`);`
10. Ketik **node module-demo.js** pada command prompt.

Third party package

1. Buatlah folder baru dengan nama **2**.
2. Buka folder tersebut dengan cmd.
3. Install **lodash** dengan mengetik **npm install lodash**.
4. Lodash adalah library JavaScript untuk membantu programmer agar lebih mudah dan rapi dalam menulis code JavaScript.
5. Sudah ada folder baru dengan nama **node_modules** dan didalamnya ada folder **lodash**.
6. Buatlah file baru dengan nama **demo.js**.
7. Ketik code berikut pada file **demo.js**:
`var _lodash = require('lodash')
console.log(_lodash.random(1, 100))`
8. Ketik **node demo.js** di cmd.
9. Install **nodemon** secara **global** dengan mengetik **npm install nodemon -g**.
10. Library **nodemon** digunakan untuk memonitor setiap perubahan yang terjadi pada file nodejs. Aplikasi akan **di-restart** otomatis oleh **nodemon**.
11. Cek folder **node_modules**, tidak ada folder nodemon karena diinstall secara global.
12. Ketik di cmd **nodemon demo.js**.
13. Setiap ada perubahan pada file **demo.js** maka akan **otomatis** dijalankan.
14. Ganti random dari 1 ke 10 lalu lihat cmd.

File package.json

1. Lanjutan dari project sebelumnya.
2. File **package.json** diperlukan jika aplikasi akan didistribusikan melalui **git repository**.
3. Jika dilakukan secara manual akan sulit karena ada ratusan bahkan ribuan file yang dependent.
4. Ketik **npm init** di cmd atau ketik **npm init --yes**.
5. Lihat di folder **2**, sudah ada file **package.json**. Biarkan default.
6. Buka file **package.json**.

IntelliJ IDEA

1. Materi NodeJS selanjutnya akan dikembangkan menggunakan IntelliJ IDEA.
2. Instalasi dapat dilihat pada url: <https://www.jetbrains.com/help/idea/running-and-debugging-node-js.html>
3. Pastikan plugins nodejs sudah terinstall. Click **Plugins**.

3. Reading, Writing, Renaming and Removing Files

Reading from file

1. Buka IntelliJ pilih **Create New Project > Web** lalu click **Next**.
2. Buat nama project: **NodeAccessFile**
3. Arahkan Project location ke **C:\Users\Kurnia\nodejs\ NodeAccessFile**.
4. Click kanan di nama project lalu **New > Directory**.
5. Buat directory dengan nama **reading**.
6. Buat file baru di package **reading** dengan nama **demo.js**.
7. Ketik code berikut di file demo.js:

```
var fs = require('fs')
```
8. Pada nodejs library **fs** digunakan untuk mengakses **file system**.
9. Selanjutnya kita butuh file untuk dibaca. Disini kita akan buat file JSON.
10. Buatlah file baru dengan nama **data.json**.
11. Ketik code berikut pada file **data.json**:

```
{  
  "jurusan" : "Informatika Unsyiah"  
}
```
12. Tutup file **data.json**.
13. Tambahkan code berikut pada file **demo.js**:

```
fs.readFile('./data.json', function(err, data) {  
  console.log(data)  
})
```
14. Run dengan click **tab Terminal** lalu ketik **cd reading** dan ketik **node demo.js**.
15. Tidak muncul sesuai dengan file data.json.
16. Edit file demo.js dengan:

```
var fs = require('fs')  
fs.readFile('./data.json', 'utf-8', function(err, data) {  
  console.log(data)  
})
```
17. Run. File data.json sudah terbaca.
18. Edit lagi file demo.js dengan:

```
var fs = require('fs')  
var data = require('./data.json')  
  
console.log(data)  
  
fs.readFile('./data.json', 'utf-8', function(err, data) {  
  console.log(data)  
})
```
19. Run.
20. Lalu edit lagi file demo.js dengan:

```
var fs = require('fs')  
var dataJson = require('./data.json')  
  
console.log(dataJson.jurusan)  
  
fs.readFile('./data.json', 'utf-8', function(err, data) {  
  console.log(data)  
})
```
21. Berhasil terbaca.
22. Lalu bagaimana jika kita ganti code dengan:

```
fs.readFile('./data.json', 'utf-8', function(err,data){
    console.log(data.jurusan)
})
```

23. Run. Muncul **undefined** karena bukan object tapi string.

24. Edit menjadi:

```
fs.readFile('./data.json', 'utf-8', function(err,data){
    var data = JSON.parse(data)
    console.log(data.jurusan)
})
```

25. Run. Sekarang data sudah bisa diakses.

26. Untuk membaca directory ketik code berikut:

```
fs.readdir('c:/', function(err, data) {
    console.log(data)
})
```

Write to a file

1. Buat directory baru dengan nama **writing**.

2. Lalu buatlah file dengan nama **write.js**.

3. Ketik code berikut di file **write.js**:

```
var fs = require('fs')

var data = {
    jurusan: 'informatika unsyiah'
}

fs.writeFile('data.json', data)
```

4. Run.

5. Muncul error.

6. Edit menjadi:

```
fs.writeFile('data.json', JSON.stringify(data))
```

7. Masih muncul error. Ini disebabkan karena masih ada parameter callback yang belum dimasukkan.

8. Edit menjadi:

```
fs.writeFile('data.json', JSON.stringify(data), function (err) {
    console.log('write finished', err)
})
```

9. File data.json berhasil digenerate. Periksa file tersebut.

Directory creation

1. Buat folder baru dengan nama **createdir**.

2. Buat file dengan nama **create-dir.js**.

3. Ketik code berikut:

```
var fs = require('fs')

fs.mkdir("coba", function (err) {
    if (err){
        console.log(err)
    }else {
        console.log("Directory created")
    }
})
```

4. Run. Directory **coba** sudah terbuat.

5. Run sekali lagi, maka akan muncul **error** karena directory coba sudah ada.

6. Edit code:

```
var fs = require('fs')

if (fs.existsSync("coba")){
    console.log("Directory already there")
}else{
    fs.mkdir("coba", function (err) {
        if (err){
```

```

        console.log(err)
    }else {
        console.log("Directory created")
    }
})
}

```

7. Run.

Renaming and Removing Files

1. Buat folder baru dengan nama **rename-remove**.
2. Buat 2 file baru di dalam directory rename-remove: **remove.js** dan **rename.js**.
3. Lalu buat sebuah directory dengan nama **lib**.
4. Di dalam folder **lib** buat 2 file baru: **notes.txt** dan **project-config.js**.
5. Ketik code berikut di file **rename.js**:

```

var fs = require('fs');

fs.renameSync('./lib/project-config.js', './lib/config.json');
console.log("Config json file renamed");

fs.rename('./lib/notes.txt', './notes.txt', function(err) {
    if (err) {
        console.log(err);
    }else {
        console.log("notes.txt berhasil dipindahkan");
    }
})

```

6. Run **rename.js**.
7. Lihat folder apakah berhasil di rename dan dipindahkan.
8. Buka file **remove.js** dan ketik code berikut:

```

var fs = require('fs');

try{
    fs.unlinkSync('./lib/config.json');
} catch(err){
    console.log(err);
}

fs.unlink('notes.txt', function (err) {
    if (err) {
        console.log(err);
    }else {
        console.log("notes.txt removed")
    }
});

```

9. Run.
10. Cek apakah file berhasil dihapus.
11. Apa yang terjadi jika file **remove.js** kita run lagi?
12. Akan muncul error, error ini didapat dari log console.
13. Ini bukan error yang di throw.
14. Jika error throw maka program akan error dan tidak dapat dijalankan lagi.

Renaming and Removing Directories

1. Buat directory dengan nama **deletedir**.
2. Buatlah file **directories.js** di dalam **deletedir**.
3. Lalu buat directory **assets** di dalam **deletedir**.
4. Di dalam **assets**, buat directory dengan nama **logs**.
5. Di dalam **logs** buat 2 buah file dengan nama **log-a.txt** dan **log-b.txt**.
6. Buka file **directories.js** dan ketik:

```
var fs = require('fs');
fs.renameSync('./assets/logs', './logs');
console.log("Directory moved");
```

7. Run.

8. Cek directory apakah sudah pindah.

9. Edit file **directories.js**:

```
var fs = require('fs');

fs.rmdir('./assets', function (err) {
  if (err) {
    throw err;
  }
  console.log("assets directory removed");
});

fs.rmdir('./logs', function (err) {
  if (err) {
    throw err;
  }
  console.log("logs directory removed")
})
```

10. Run.

11. Muncul error. Folder **logs** tidak bisa dihapus karena tidak kosong.

12. Tapi folder **assets** berhasil dihapus.

13. Edit **directories.js**:

```
var fs = require('fs');

fs.readdirSync('./logs').forEach(function (fileName) {
  fs.unlinkSync('./logs/' + fileName);
})

fs.rmdir('./logs', function (err) {
  if (err) {
    throw err;
  }
  console.log("logs directory removed")
})
```

14. Run.

15. Sekarang directory **logs** berhasil dihapus.